

WORKSHOP

Agentic Coding

Grundlagen, Open-Source-Lösungen und lokale Modelle

Gregor Wolf

Ablauf

Input

45 min

Fragen

15 min

Hands-on

90 min

Diskussion

30 min

Bevor wir loslegen ...

Bevor wir loslegen ...

„Wer hat schon einmal mit KI im Browser geschattet?“

Bevor wir loslegen ...

„Wer hat schon einmal mit KI im Browser geschattet?“

„Wer hat schon einmal eine Agent-App benutzt, etwa Claude Code, Cowork oder Design?“

Bevor wir loslegen ...

„Wer hat schon einmal mit KI im Browser geschattet?“

„Wer hat schon einmal eine Agent-App benutzt, etwa Claude Code, Cowork oder Design?“

„Wer hat schon einmal ein LLM lokal ausgeführt?“

Bevor wir loslegen ...

„Wer hat schon einmal mit KI im Browser geschattet?“

„Wer hat schon einmal eine Agent-App benutzt, etwa Claude Code, Cowork oder Design?“

„Wer hat schon einmal ein LLM lokal ausgeführt?“

„Wer weiß, was ein Mixture-of-Experts-Modell ist?“

INPUT

Sprach-KI → LLM

INPUT

KONTEXT

Der schönste Stadtteil
Leipzigs ist

Neuronales
Netz



OUTPUT

WAHRSCHEINLICHKEIT FÜR DAS
NÄCHSTE TOKEN

Connewitz		68 %
Plagwitz		12 %
Gohlis		6 %
Reudnitz		0,3 %
...		

INPUT

Sprach-KI → LLM

INPUT

KONTEXT

Der schönste Stadtteil
Leipzigs ist

Neuronales
Netz



„Inferenz ...“

OUTPUT

WAHRSCHEINLICHKEIT FÜR DAS
NÄCHSTE TOKEN

Connewitz		68 %
Plagwitz		12 %
Gohlis		6 %
Reudnitz		0,3 %
...		

INPUT

Sprach-KI → LLM

INPUT

KONTEXT

Der schönste Stadtteil
Leipzigs ist

Neuronales
Netz



„Inferenz ...“

OUTPUT

WAHRSCHEINLICHKEIT FÜR DAS
NÄCHSTE TOKEN

Connewitz		68 %
Plagwitz		12 %
Gohlis		6 %
Reudnitz		0,3 %
...		

💡 LLMs sind riesige neuronale Netze mit vielen Millionen bis Trillionen Parametern.

INPUT

Tokens

Der	schönste	Stadt	teil	Leipzig	s	ist	Connewitz	.
-----	----------	-------	------	---------	---	-----	-----------	---

💡 Tokens sind quasi "Wörter" für LLMs

💡 Tokenzahl $\approx$ Anzahl Wörter $\times$ 1,5 oder Zeichenzahl / 4

INPUT

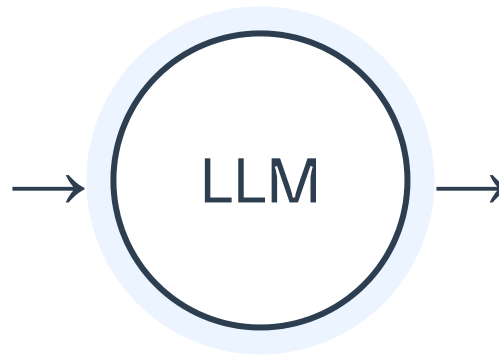
Chatten mit LLMs

INPUT

KONTEXT

user: Schreibe eine Aussage
über den schönsten Stadtteil
Leipzigs.

assistant:



OUTPUT

WAHRSCHEINLICHKEIT FÜR DAS
NÄCHSTE TOKEN

Connewitz		68 %
Plagwitz		12 %
Gohlis		6 %
Reudnitz		0,3 %
...		

INPUT

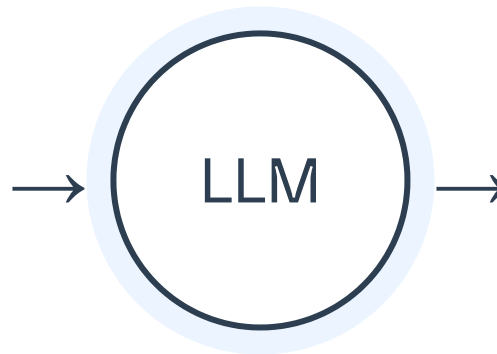
Chatten mit LLMs

INPUT

KONTEXT

user: Schreibe eine Aussage
über den schönsten Stadtteil
Leipzigs.

assistant: Connewitz



OUTPUT

WAHRSCHEINLICHKEIT FÜR DAS
NÄCHSTE TOKEN

ist		54 %
gilt		16 %
liegt		8 %
hat		6 %
...		

INPUT

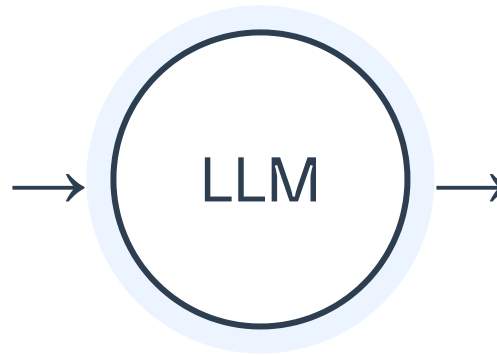
Chatten mit LLMs

INPUT

KONTEXT

user: Schreibe eine Aussage
über den schönsten Stadtteil
Leipzigs.

assistant: Connewitz ist



OUTPUT

WAHRSCHEINLICHKEIT FÜR DAS
NÄCHSTE TOKEN

der		42 %
bekannt		19 %
ein		11 %
besonders		7 %
...		

Training



- 💡 Training erfolgt ausschließlich vor der Benutzung.
- 💡 Fähigkeiten des Modells hängen von den Trainingsdaten ab.
- 💡 Modellverhalten basiert auf den Präferenzen des Herstellers.

INPUT

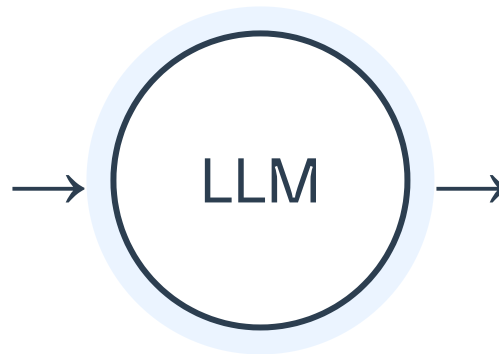
Thinking

INPUT

KONTEXT

system: Denke nach, bevor du antwortest.

user: Plane die Migration in drei Schritten.



STRUCTURED OUTPUT

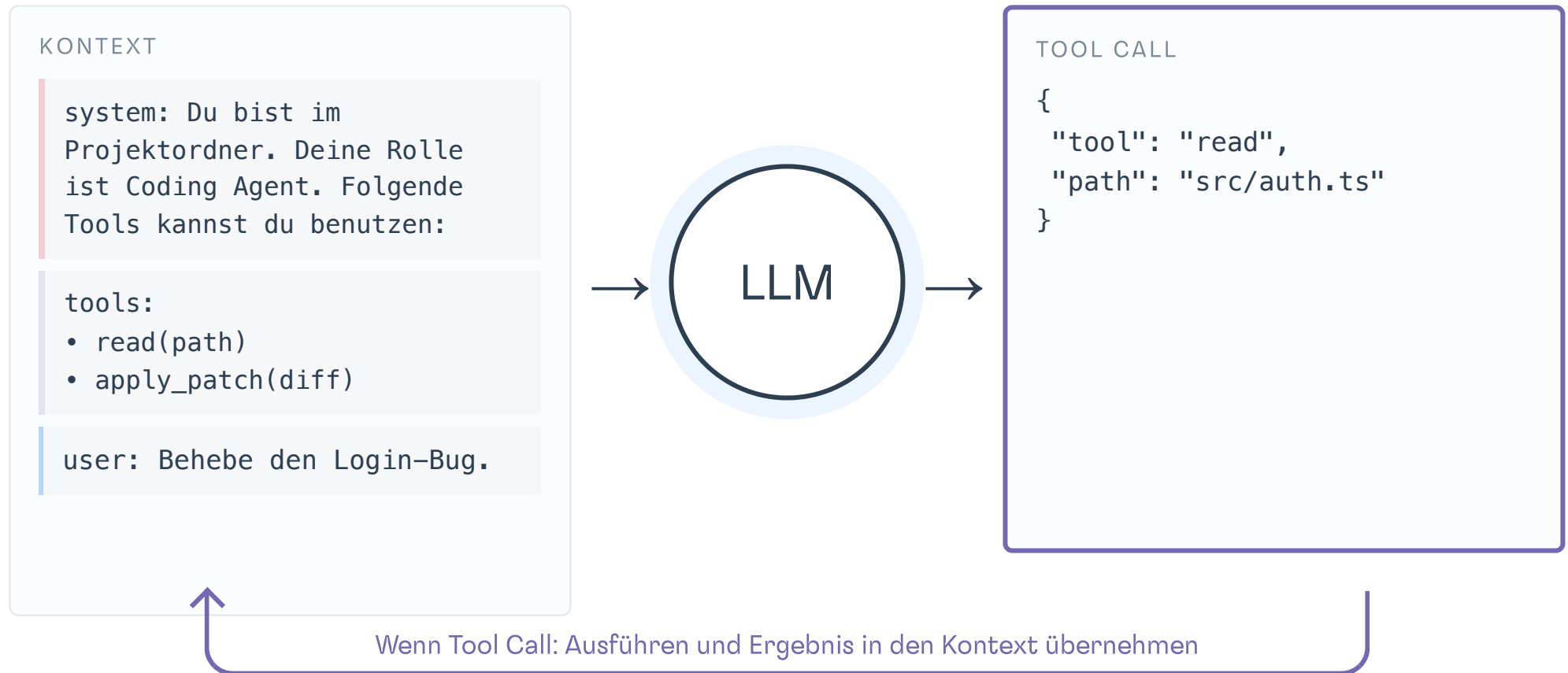
```
<think>
Abhängigkeiten prüfen ...
Rollback definieren ...
zuerst Staging ...
</think>
```

```
assistant: 1. Inventur der ...
```


INPUT

Agents

INPUT

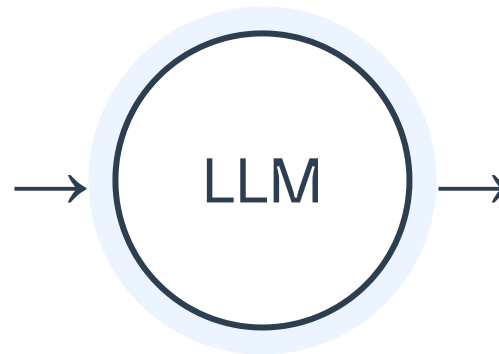


💡 Agenten sind LLMs, die mithilfe von Tools handeln, also agieren können.

INPUT

Agents

INPUT



STRUCTURED OUTPUT



Wenn Tool Call: Ausführen und Ergebnis in den Kontext übernehmen

💡 Agenten sind LLMs, die mithilfe von Tools handeln, also agieren können.

Agent Harness

INPUT

KONTEXT

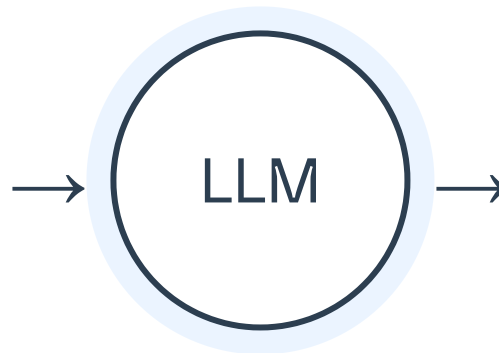
system: Du bist im Projektordner. Deine Rolle ist Coding Agent. Folgende Tools kannst du benutzen:

tools:

- read(path)
- apply_patch(diff)

user: Behebe den Login-Bug.

tool: src/auth.ts gelesen



STRUCTURED OUTPUT

THINKING + TOOL CALL

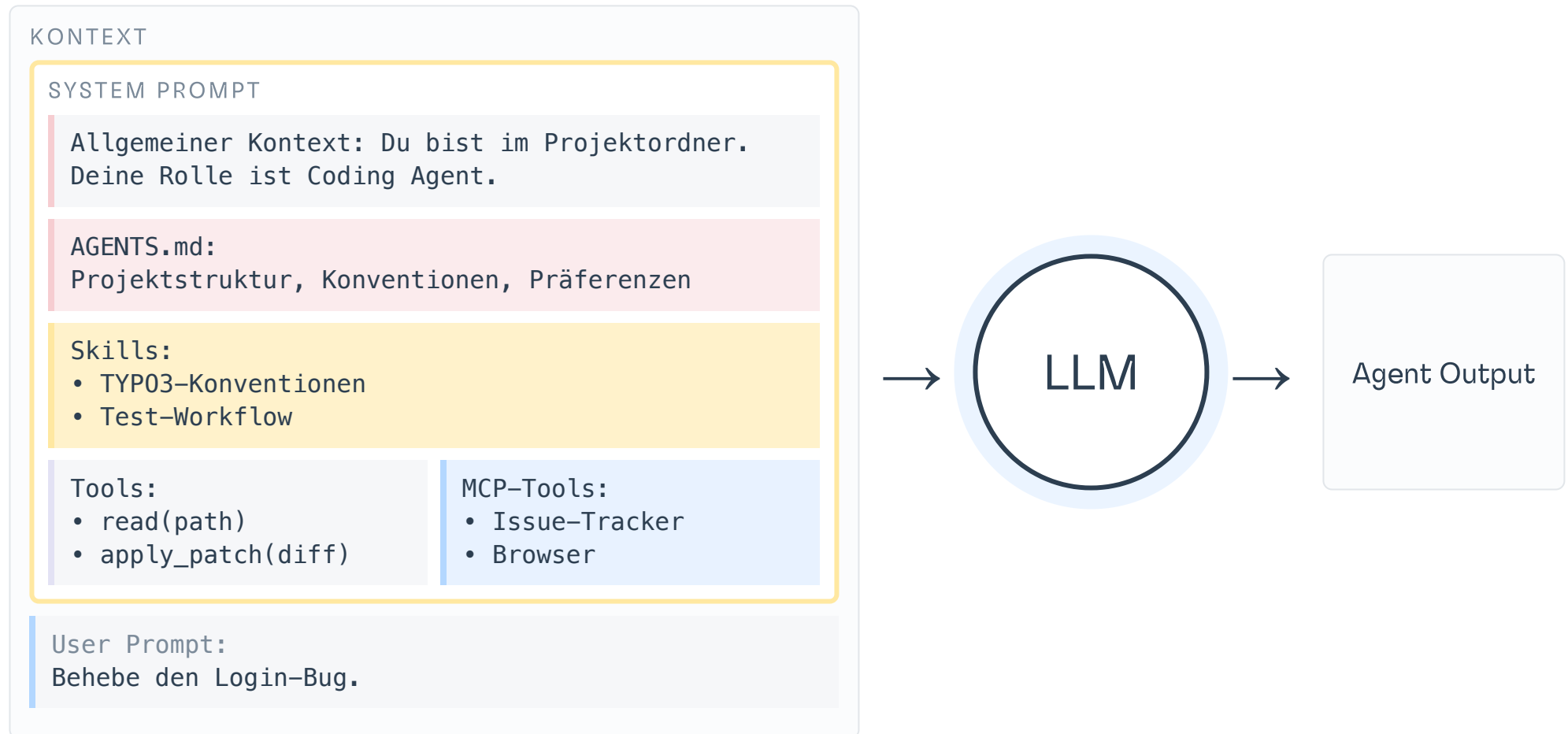
Login-Prüfung korrigieren ...

```
{  
  "tool": "apply_patch",  
  "path": "src/auth.ts"  
}
```

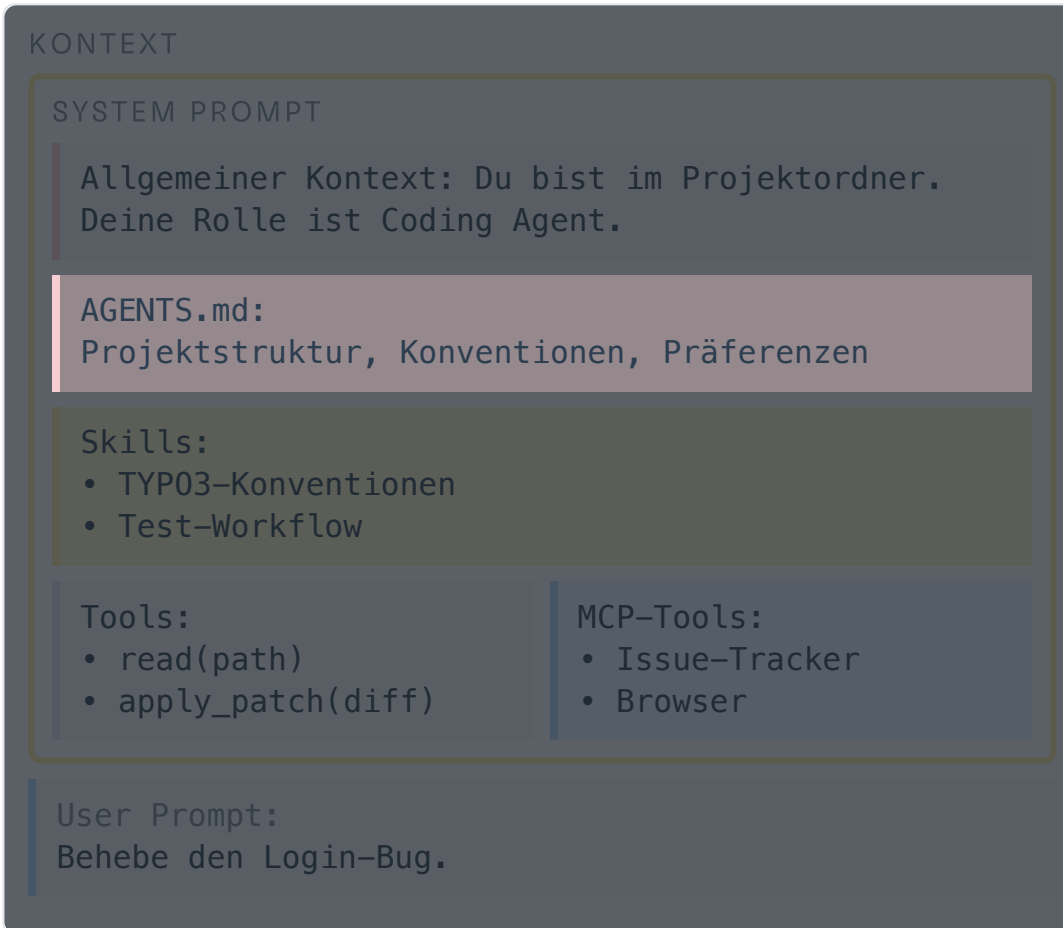
Wenn Tool Call: ausführen und Ergebnis in den Kontext übernehmen

💡 Der Harness verwaltet Kontext, Werkzeuge, Berechtigungen und Ausführung.

Context Engineering



Context Engineering

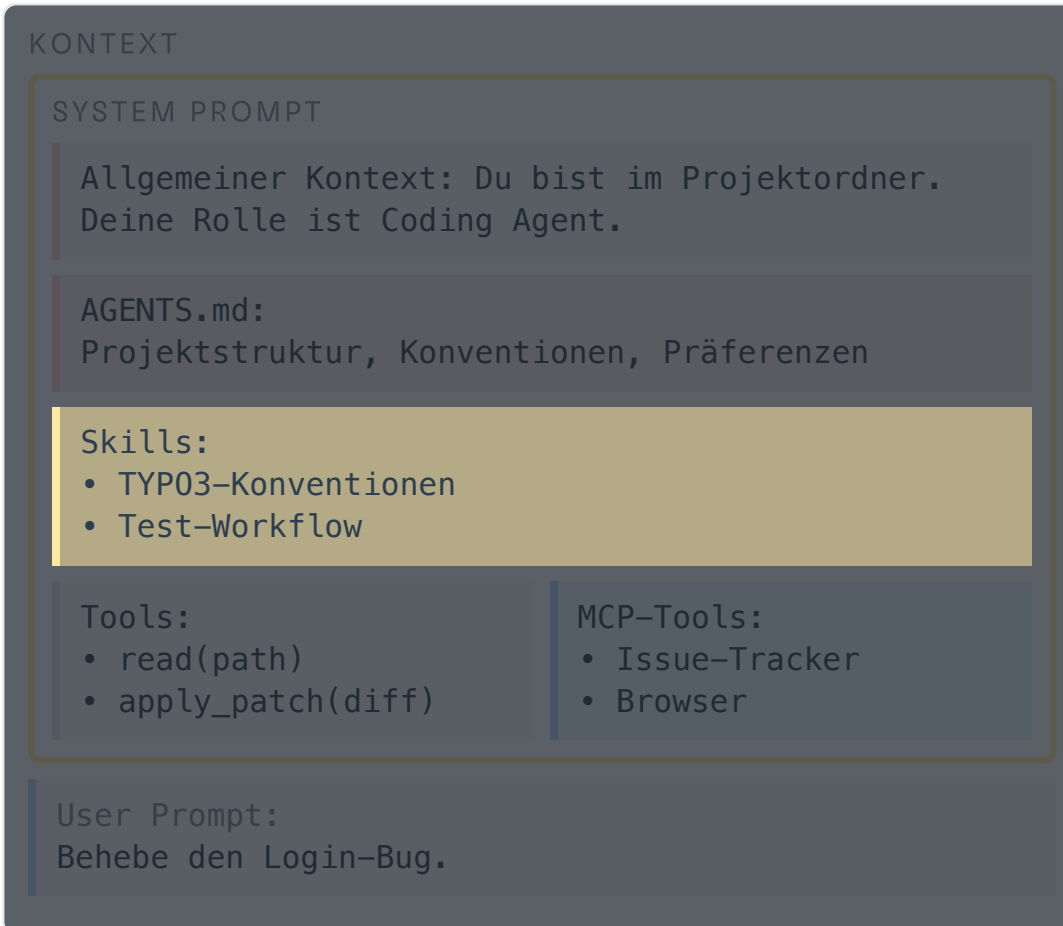


- Projektregeln und -kontext für den Agenten
- Global oder projektspezifisch
- Entspricht der CLAUDE.md bei Claude Code



Veränderter Kontext je nach Projekt

Context Engineering

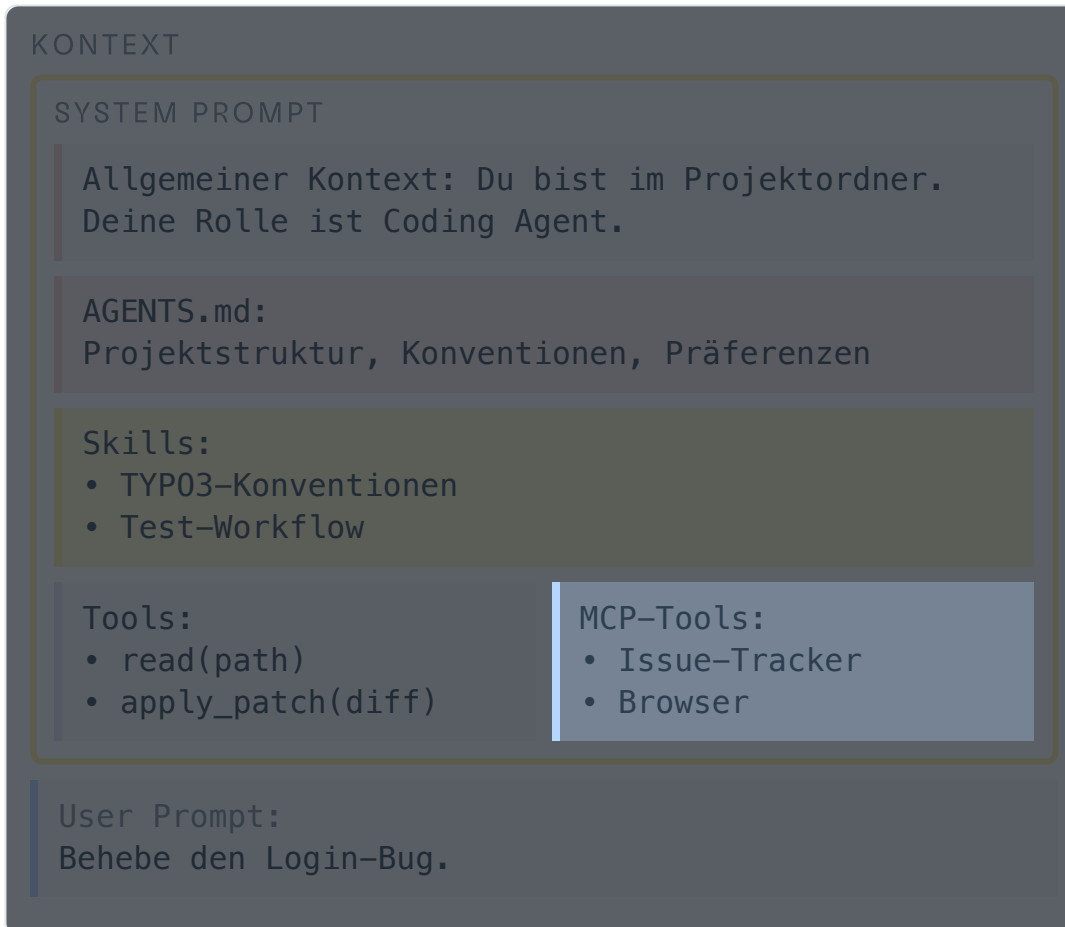


- Im Grunde Markdown-Dateien mit Anleitungen
- Werden bei Bedarf in den Kontext geladen
- Können Dateien, Skripte und Vorlagen mitbringen



Erweitern Kontext nicht nach Projekt, sondern nach Aufgabe

Context Engineering



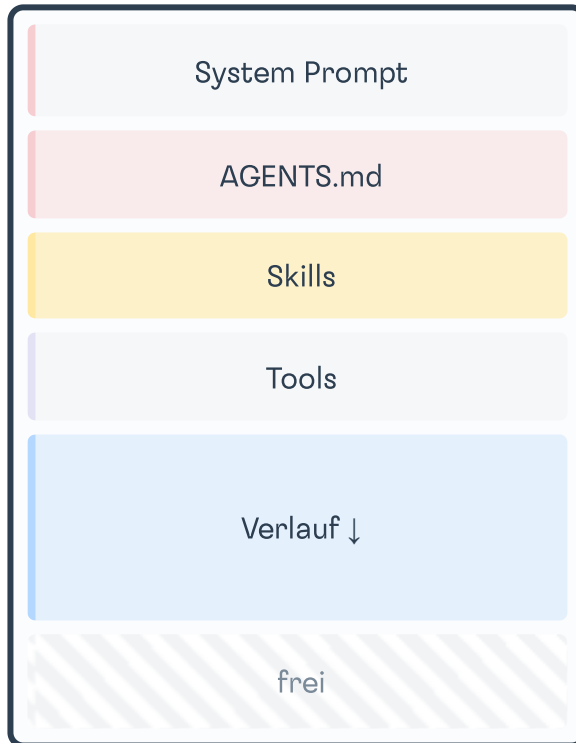
- Standardisierte Schnittstelle zu externen Systemen
- Stellt Tools und zusätzliche Daten bereit
- Zum Beispiel Jira, Confluence oder Browser



Gibt Agent zusätzliche Tools zu externen Programmen

Context Window

KONTEXTFENSTER · FESTE GRÖSSE



LLMs haben begrenzte Kontextfenster-Größe (Top-Modelle aktuell: 1 Million Tokens)

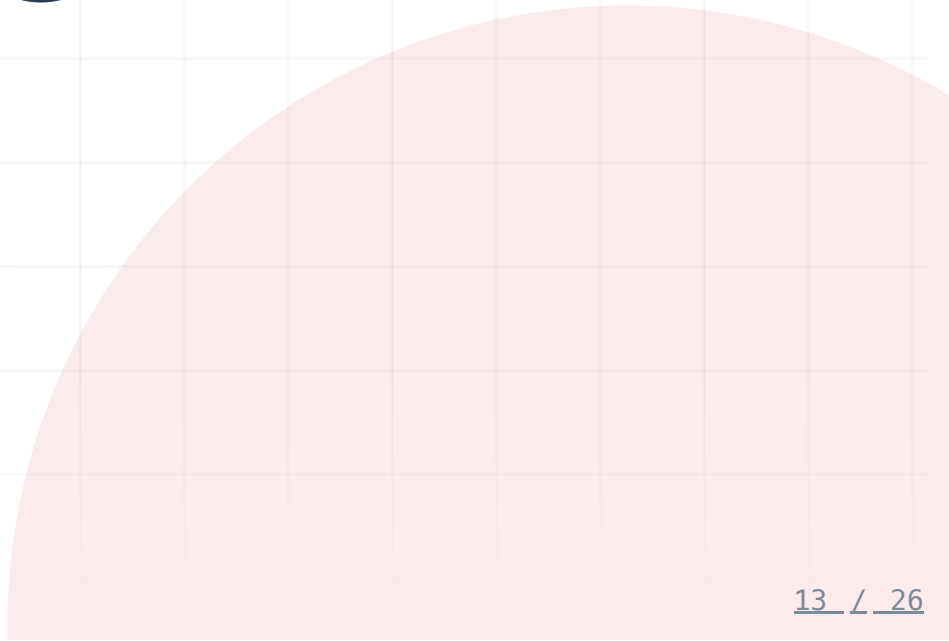
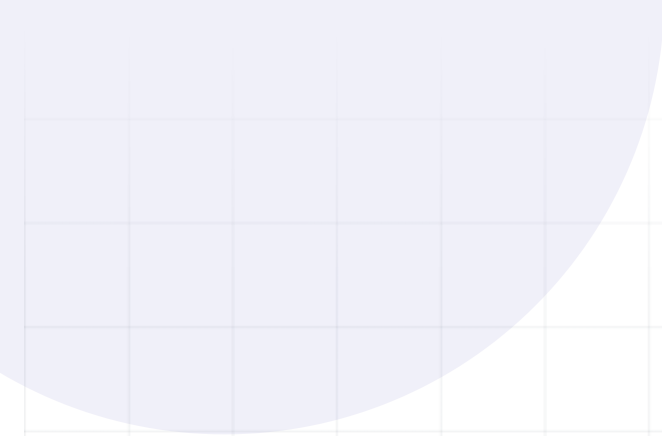


Wegen Lost-In-The-Middle: Kontext $\leq 150k$ Tokens halten

→ AGENTS.md klein halten

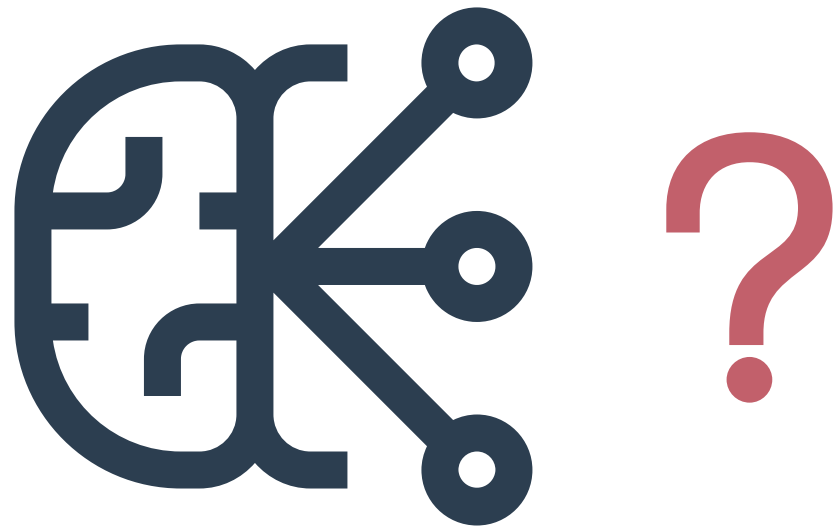
→ Detailanweisungen in Skills auslagern

→ Wenn Context-Window voll: /handoff und neuer Chat oder /compact



DEMO

Missverständnis Intelligenz



In die Praxis! lokal und Open-Source

Wir brauchen:

- Lokal laufendes Sprach-Modell 
- Harness-Anwendung 

Lokale Modelle

1. Modell muss in den Grafikspeicher (VRAM) passen, RAM wäre zu langsam

→ Ausnahme Apple Silicon: Shared Memory → VRAM $\approx$ RAM

$$\text{VRAM-Verbrauch} \approx \text{Anzahl Modellparameter} \times \text{Bits pro Parameter} \div 8 + \text{Context-Window-Size}$$

abhängig von Quantisierung meist 8 oder 4

2. Muss schnell genug laufen: gemessen in Tokens/Sekunde

Zusätzliche Faktoren:

- Dense / Mixture of Experts (MoE)
- Auf Mac: MLX-Unterstützung
- Quantisierung
- Multi Token Prediction (MTP)

LLM-Runtimes



Ollama

- + Etablierter Standard
- + Einfache Anbindung
- + Terminal und API als Hauptanwendung
- Keine Unterstützung bei der Modellwahl



LM Studio

- + Chatinterface mit vielen Funktionen
- Umständliche Anbindung
- + Fokus auf benutzerfreundliche GUI
- + Gute Modellvorauswahl

LLM-Runtimes



```
~ ollama run gemma4:e2b  
pulling manifest  
success
```

```
>>> Was ist ein Coding Agent?
```

```
Ein Coding Agent ist ein Sprachmodell, das über Tools  
Dateien liest, Änderungen schreibt und Befehle ausführt.
```

```
>>> |
```

Chats

File System Project

783 tokens

Secret project

Ascii

Investigate EOS

Intelligence test

openai/gpt-oss-20b

Please write an example self contained toy filesystem in C++

Thought for a brief moment

cpp

```
// toy_fs.cpp
#include <iostream>
#include <memory>
#include <unordered_map>
#include <vector>
#include <string>
#include <sstream>

class File {
public:
    explicit File(std::string name) : name_(std::move(name)) {}
    const std::string& name() const { return name_; }

    void write(const std::string& data) { content_ = data; }
    const std::string& read() const { return content_; }

private:
    std::string name_;
    std::string content_;
};
```

Copy

Send a message to the model...

Reasoning Effort

js-code-sandbox

Input token count: 0 Tokens: 783/131072

Context

Model

Program

Preset

Discard Unsaved

Code guidelines

Commit Changes

Save As New...

System Prompt

Editor

- Only smart pointers
- No template meta-programming unless very minimal and necessary
- no explanations, just write the code

Token count: 25

LM Studio 0.3.24 (Build 2) User Power User Developer RAM: 17.47 GB | CPU: 0.00 %

Modell und Hardwarewahl Empfehlungen

Lokal:

- LM Studio Empfehlungen
- willitrun.ai LLM-Empfehlungen basierend auf Hardware
- local.ai Gute Analysen zu Hardware und Benchmarks
- llmconfigurator.com GPU-Kaufberatung und generell gute Infos

Generell:

- livebench.ai Nur für große Modelle
- artificialanalysis.ai Viele Modelle und Compare-Funktion

Search local models by name or author...

Staff picksGGUF, MLXBest Match



Bonsai 27B

Bonsai is a 27B model, but takes only about ... 22 days ago



Gemma 4 12B QAT

Gemma 4 12B optimized with Quantized Awa... 78 days ago



Gemma 4 12B

The new Gemma 4 12B Unified reasoning mo... 80 days ago



Gemma 4 E4B

Gemma4, effective 4B version. Supports im... 133 days ago



Gemma 4 E2B

Gemma4, effective 2B version. Supports im... 133 days ago



Nemotron 3 Nano 4B

General purpose reasoning and chat model ... 159 days ago



Qwen3.5 9B

Qwen3.5 represents a significant leap forwa... 172 days ago



Lfm2 24B A2B

LFM2 is a family of hybrid models designed ... 179 days ago



Lfm2.5 1.2B

LFM2.5 is a new family of hybrid models de... 200 days ago

prism-ml/bonsai-27b

484,21543Last updated: 22 days agoStaff Pick

Bonsai is a 27B model, but takes only about 4GB. Capable of reasoning, coding, vision, and tool use.

PARAMS 27B

ARCH qwen35

DOMAIN llm

FORMAT GGUFMLX

CAPABILITIES

Vision

Tool Use

Reasoning

Download Options

MLX

Ternary Bonsai 27B

2BIT

8.52 GB

Full GPU Offload Possible

Download8.52 GB

README

Bonsai 27B

Bonsai 27B is PrismML's family of binary and ternary Qwen3.6 27B models, designed to retain strong reasoning, coding, and agentic capabilities at exceptionally small footprints.

Highlights

- 27B reasoning in a phone-class footprint: binary weights compress the language model to about 3.9 GB, bringing 27B-class local inference to high-end phones.
- Quality that survives extreme compression: across 15 thinking-mode benchmarks, the binary model retains 89.5% of FP16's average score at 1.125 effective bits per weight; the ternary model retains 94.6% at 1.71 bits.

18 / 26

Modell und Hardwarewahl

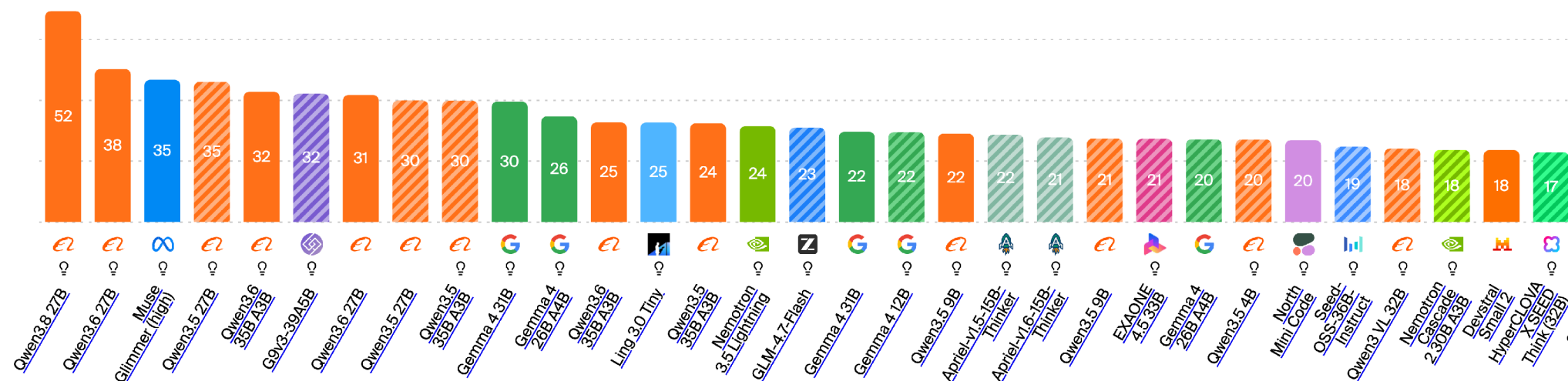
Artificial Analysis Intelligence Index

Open Source, Tiny, Small Models

Artificial Analysis Intelligence Index v4.1.1 incorporates 9 evaluations: GDPval-AA v2, τ^3 -Banking, Terminal-Bench v2.1, SciCode, Humanity's Last Exam, GPQA Diamond, CritPt, AA-Omniscience, AA-LCR

■ Estimate (independent evaluation forthcoming)

Artificial Analysis



💡 Reasoning models are indicated by a lightbulb icon

Open Source Harness



OpenCode

- + Sehr etabliert
- + Robust und kein Slop
- + Funktioniert mit vielen Modellen

- Viele Funktionen von Claude Code fehlen
- Plugin-Ökosystem ist nicht so groß
- Keine Agent-Sandboxes → OpenCode TUI in Container starten



OpenChamber

Nutzt OpenCode als Backend

Viele zusätzliche Features und besseres UI



OpenWork

Claude Cowork Alternative

Open Source Harnesses



Hermes

github.com/nounsresearch/hermes-agent



Pi (Oh My Pi)

github.com/earendil-works/pi
github.com/can1357/oh-my-pi



Deepseek Harness

github.com/deepseek-ai/deepseek-harness

Meta Harnesses



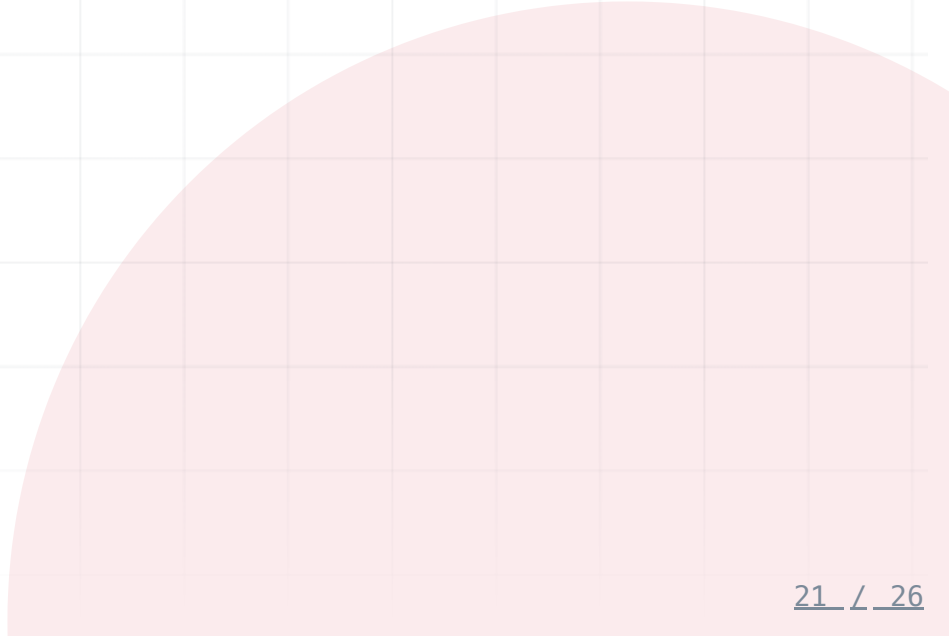
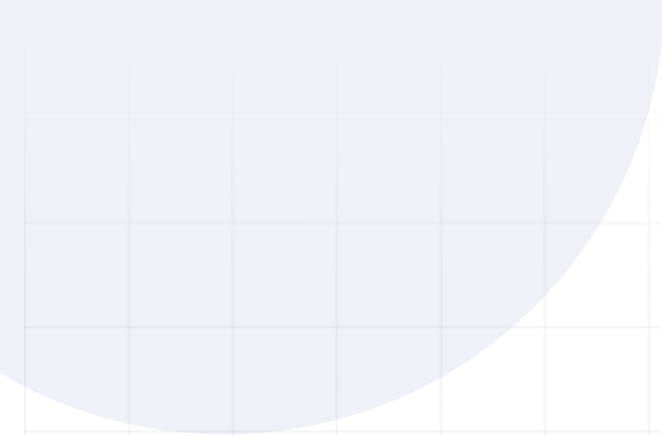
CrewAI

crewai.com



Omnigent

omnigent.ai



Fragen?



HANDS-ON!

Ab ins Repo! 🚀

- OpenCode, OpenChamber oder OpenWork ausprobieren
- Lokales Modell mit Ollama einrichten
- Atlassian MCP anbinden
- Skills finden und eigene Skills erstellen
- Live-Demos lokal ausprobieren



urlr.me/AFqu4Q

OpenCode Cheatsheet

<code>/connect</code>	Provider verbinden
<code>/models</code>	Modell auswählen
<code>/init</code>	AGENTS.md erstellen
<code>Tab</code>	Zwischen Plan- und Build-Modus wechseln
<code>/undo /redo</code>	Änderungen zurück oder vor
<code>@</code>	Dateien referenzieren
<code>/sessions</code>	Sessions verwalten

Ausführlicher Guide:

→ ai.sulat.com/the-definitive-guide-to-opencode-from-first-install-to-production-workflows-aae1e95855fb

KI Sicherheit

- ⚠ Wenn möglich Sandboxing verwenden
- ⚠ Agent immer im Projektordner und nicht im Benutzerordner starten
- ⚠ Sich bewusst machen, an wen man gerade welche Daten schickt
- ⚠ Nur nötige Berechtigungen geben: Plan-Modus verwenden und OpenCode-Berechtigungen anpassen
- ⚠ Ergebnis händisch validieren und vor dem Coding Tests schreiben (lassen)

Diskussion